

Detector de Anomalias em Métricas de Servidores

Detecção não supervisionada de comportamento anômalo em séries temporais multivariadas — dataset SMD

PROBLEMA

Cada servidor publica 38 métricas ao mesmo tempo. Degradações que só aparecem na combinação dos sinais passam despercebidas até virarem incidente.

MOTIVAÇÃO

O monitoramento hoje depende de limiar fixo por métrica e inspeção manual de painéis: alerta demais no trivial e tarde no que importa.

PÚBLICO-ALVO

Analista de Operações / SRE (recebe o sinal e faz a triagem) e Coordenador de Observabilidade (decide manter, recalibrar ou retreinar o detector).

OBJETIVO

Sinalizar automaticamente as janelas de tempo em que a máquina se afasta do padrão normal que ela mesma exibiu no passado.

PAPEL DA IA

Um autoencoder aprende a reconstruir o comportamento normal. O que ele reconstrói mal é o que não se parece com o normal — esse erro é o escore de anomalia.

EQUIPE — GRUPO 12

Breno Alexandre de Albuquerque Santos

Erasmus de Melo Gusmão

Gabriel Victor Alves Santana

João Mateus Queiroz Moreira

João Pedro da Silva Rodrigues

Leonardo Canuto de Oliveira Magalhães

Orlando Gomes dos Reis Neto

Pedro Oliveira Pessoa Guerra

REPOSITÓRIO

github.com/ErasmusGusmao/detector-anomalias-smd

Dados e solução proposta

28.479

amostras de treino
(só comportamento normal)

28.479

amostras de teste
(com anomalias rotuladas)

38

métricas por instante
(atributos de entrada)

9,46%

taxa base de anomalia
(2.694 instantes)

BASE DE DADOS

SMD — Server Machine Dataset

Origem: coletado do trabalho OmniAnomaly (NetManAIOps) e publicado no Kaggle. São métricas reais de operação de servidores, anonimizadas.

Dimensão original: 28 máquinas × 38 métricas, cada máquina com sua própria série temporal completa.

Recorte do projeto: a máquina machine-1-1, definida em config.py. Uma máquina já é uma série multivariada completa — o recorte reduz custo sem descaracterizar o problema.

Arquivos: train (metade inicial, sem anomalia) · test (metade final) · test_label (rótulo 0/1 por instante, usado só na avaliação).

SOLUÇÃO PROPOSTA

Autoencoder treinado só com o normal

ENTRADA → janela de 50 instantes consecutivos × 38 métricas padronizadas (z-score), deslocando de 1 em 1.

SAÍDA → (a) escore de anomalia = erro de reconstrução da janela; (b) classificação binária normal / anômala.

VARIÁVEL PREVISTA → o rótulo binário da janela. Repare: o rótulo NÃO entra no treino, só na avaliação — por isso a abordagem é não supervisionada.

DECISÃO → a janela é anômala quando o escore ultrapassa um limiar derivado do percentil dos erros do próprio conjunto de treino.

Requisitos funcionais

Elicitados com o framework GR4ML: cada RF nasce de um Decision Goal + Question Goal da Business View — a origem está registrada no documento, e é ela que dá rastreabilidade.

RF	O sistema deve...	Status no protótipo
RF-01	calcular um escore de anomalia para cada janela de 50 instantes, a partir do erro de reconstrução das 38 métricas padronizadas.	Atendido
RF-02	classificar cada janela como normal ou anômala, comparando seu escore ao limiar vigente.	Atendido
RF-03	disponibilizar, para cada janela anômala, o erro decomposto por métrica, ordenado por contribuição.	Do sistema-alvo
RF-04	aprender o padrão normal da máquina a partir da partição de treino, sem usar rótulos.	Atendido
RF-05	persistir o modelo treinado, o limiar de anomalia e as estatísticas de padronização.	Parcial
RF-06	derivar o limiar do percentil configurado sobre os escores de treino, registrando o valor aplicado.	Atendido
RF-07	calcular e registrar precisão, recall e F1 da versão avaliada, contra os rótulos de teste.	Parcial

Onde o protótipo ainda não chega — e por quê

RF-03 e RF-05/RF-07 parciais: o erro hoje é agregado por janela (falta a decomposição por métrica); a média e o desvio da padronização não são persistidos junto do modelo; e o percentual de janelas sinalizadas não vai para o results.json. São lacunas conhecidas, declaradas no documento de arquitetura.

Requisitos não funcionais e critérios de aceitação

RNF	Qualidade exigida	Como verificamos	Resultado medido
RNF-01	Baixa taxa de falso positivo, medida pela precisão, acima da taxa base do conjunto.	Executar a avaliação no teste e comparar a precisão com a taxa base (9,46%).	25,3% > 9,46%
RNF-02	Localizar as métricas responsáveis pela anomalia (erro por dimensão).	Verificar se a saída traz o erro decomposto por métrica.	Não atendido
RNF-03	Custo de treino viável: concluir o treino sem GPU dedicada.	Executar o pipeline em CPU e medir o tempo.	165 s em CPU
RNF-04	Cobrir degradações que só aparecem na combinação das métricas (recall).	Medir o recall no teste e comparar com um detector de limiar por métrica.	recall 88,0%

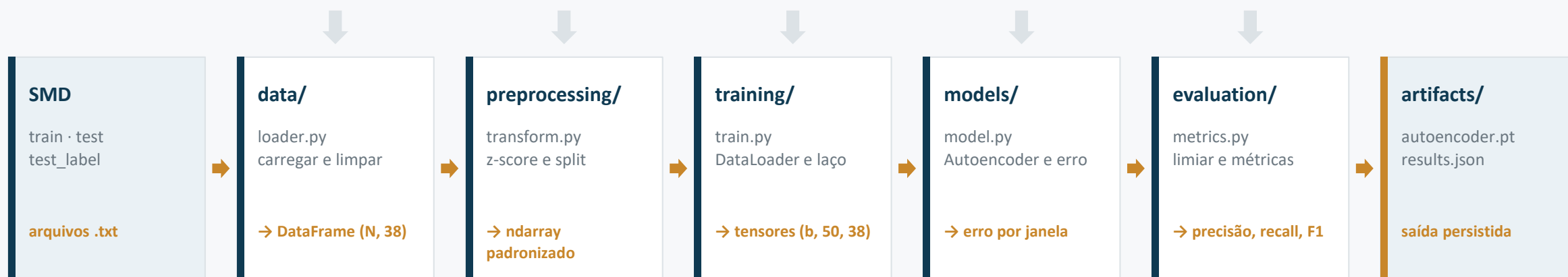
CRITÉRIO DE ACEITAÇÃO DO CONJUNTO

“Este conjunto de requisitos será bem-sucedido se o detector sinalizar ao menos 85% dos eventos anômalos conhecidos do SMD (recall), mantendo precisão superior à taxa base de anomalias do conjunto avaliado (9,46% na machine-1-1), verificado a cada nova versão do modelo.”

ATENDIDO na execução de referência: recall 88,0% ≥ 85% · precisão 25,3% > 9,46%

Arquitetura do sistema

main.py — orquestrador: é o único que conhece a ordem das etapas



src/utils/ — apoio transversal (não carrega dados do pipeline)

`config.py` — caminhos e hiperparâmetros
`torch_utils.py` — device e sementes

REGRA DE DEPENDÊNCIA: as setas nunca voltam

Nenhum pacote de `src/` importa o `main.py`, e `utils/` não importa ninguém do pipeline. É o que permite usar o pré-processamento ou as métricas fora do pipeline — nos testes, por exemplo — sem arrastar o resto junto.

Decisões arquiteturais

D1 · Um pacote por etapa do pipeline

Cada etapa tem um dono e um arquivo de teste próprio. Foi o que permitiu várias pessoas trabalharem em paralelo sem conflito de merge.

SEPARAÇÃO DE RESPONSABILIDADES

D2 · Pré-processamento em NumPy, antes de qualquer tensor

Mantém data/ e preprocessing/ executáveis sem PyTorch instalado — destravou colegas cujo ambiente não tinha a biblioteca.

BAIXO ACOPLAMENTO

D3 · Configuração centralizada em utils/config.py

Trocar janela, épocas ou percentil é editar uma linha, não caçar constante espalhada. É o que tornou viável comparar configurações.

MANUTENIBILIDADE

D4 · Padronização com estatísticas só do treino

Validação e teste usam a média e o desvio do treino. O contrário vazaria informação do teste para o pré-processamento.

VALIDADE DO EXPERIMENTO

D5 · Split temporal, nunca aleatório

A série é temporal: embaralhar antes de dividir treinaria o modelo com o futuro e inflaria o resultado artificialmente.

VALIDADE DO EXPERIMENTO

D6 · Persistência separada da definição do modelo

persistence.py grava só os pesos; a arquitetura fica em model.py. Carregar exige recriar a estrutura — não dependemos de serializar a classe.

ALTA COESÃO

D7 · Orquestração exclusivamente no main.py

Os módulos não sabem em que ordem são chamados. Cada função pode ser exercitada isolada num teste.

TESTABILIDADE

D8 · Métricas em NumPy puro, sem scikit-learn

Uma dependência a menos e o cálculo de precisão e recall fica auditável linha a linha — e testável.

TESTABILIDADE

Estrutura do projeto

```

detector-anomalias-smd/
├── main.py                # ponto único de execução do pipeline
├── requirements.txt
├── README.md
├── src/
│   ├── data/loader.py    # carregamento e limpeza (pandas)
│   ├── preprocessing/transform.py # padronização e split (NumPy)
│   ├── models/
│   │   ├── model.py      # Autoencoder (nn.Module) e erro
│   │   └── persistence.py # salvar / carregar state_dict
│   ├── training/train.py  # DataLoader, laço de treino e validação
│   ├── evaluation/metrics.py # limiar, predição e métricas
│   └── utils/
│       ├── config.py     # caminhos e hiperparâmetros
│       └── torch_utils.py # device e sementes
├── tests/
├── docs/
├── apresentacao/
├── data/
└── artifacts/

```

4 arquivos, um por pacote de src/

requisitos (GR4ML) e arquitetura

slides, gerados a partir dos resultados

SMD (não versionado)

modelo e resultados gerados

Um diretório por responsabilidade

O nome da pasta diz a etapa do pipeline. Quem chega no repositório sabe onde procurar sem abrir arquivo.

tests/ espelha src/ na mesma granularidade

Um arquivo de teste por pacote: test_data, test_preprocessing, test_model, test_training. É imediato saber onde um comportamento é verificado.

O que é gerado fica fora do versionamento

data/ e artifacts/ não vão para o Git: dados brutos e modelos treinados são reproduzíveis, não código-fonte.

main.py na raiz, sozinho

Um único ponto de entrada: python main.py roda tudo, do arquivo bruto ao results.json.

Responsabilidades dos módulos

Módulo	Responsabilidade	Entrada → saída
data/loader.py	Ler os arquivos do SMD e remover linhas inválidas preservando o alinhamento entre métricas e rótulos.	caminho → DataFrame
preprocessing/transform.py	Padronizar por z-score e dividir a série preservando a ordem temporal.	ndarray → ndarray
models/model.py	Definir o autoencoder (nn.Module, forward) e calcular o erro de reconstrução por janela.	tensor → tensor
models/persistence.py	Salvar e carregar os pesos (state_dict) do modelo treinado.	modelo ↔ arquivo .pt
training/train.py	Montar TensorDataset/DataLoader e executar o laço de treino e validação.	matrizes → modelo + histórico
evaluation/metrics.py	Derivar o limiar, binarizar os erros e calcular precisão, recall, F1 e acurácia.	erros + rótulos → métricas
utils/config.py	Fonte única de caminhos e hiperparâmetros do projeto.	— (constantes)

```
# main.py - a integracao, em ordem
train_clean = clean_data(load_data(config.TRAIN_PATH))
X_train, X_val, _, _ = split_data(X_full, time_index, test_size=0.2)
X_train, mean, std = standardize(X_train)
X_val, _, _ = standardize(X_val, mean=mean, std=std)
model = create_model(input_dim=X_train.shape[1], device=device)
resultado = train_model(model, X_train, X_val, device=device)
save_model(model)
threshold = reconstruction_threshold(err_train, ANOMALY_PERCENTILE)
metrics = calculate_metrics(y_aligned, predict_anomalies(err, threshold))
```

Como o main.py integra

Ele chama uma função de cada módulo, na ordem, e passa o resultado adiante. Nenhum módulo chama outro: quem conhece a sequência é só o orquestrador.

Na linha destacada está a decisão D4 em ação — a validação é padronizada com a média e o desvio vindos do treino.

Código Python, NumPy e tipagem

src/preprocessing/transform.py

```
def standardize(
    X: np.ndarray,
    mean: np.ndarray | None = None,
    std: np.ndarray | None = None,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Padroniza os atributos para media 0 e desvio 1."""
    X_float = np.asarray(X, dtype=float)

    if mean is None or std is None:
        mean = np.mean(X_float, axis=0)
        std = np.std(X_float, axis=0)

    safe_std = np.where(std == 0, 1.0, std)
    return (X_float - mean) / safe_std, mean, std
```

ENTRADA / SAÍDA matriz (n_amostras, 38), opcionalmente com média e desvio já calculados → tupla com a matriz padronizada + a média + o desvio usados.

NUMPY np.mean e np.std com axis=0 calculam por coluna, ou seja, por métrica. A subtração e a divisão são vetorizadas: uma operação sobre a matriz inteira, sem laço.

TIPAGEM o | None diz que os parâmetros são opcionais; o tipo de retorno declara as três saídas. Quem chama não precisa ler o corpo da função.

RESPONSABILIDADE faz UMA coisa: padroniza. É a mesma função em dois modos — calcula as estatísticas (treino) ou aplica as recebidas (validação e teste).

src/models/model.py

```
def reconstruction_error(
    model: Autoencoder,
    X: np.ndarray,
    window_size: int = config.WINDOW_SIZE,
    device: str = config.DEVICE,
) -> np.ndarray:
    """Erro de reconstrucao (MSE) por janela."""
    X_array = np.asarray(X, dtype=np.float32)
    ...
    windows = sliding_window_view(
        X_array, window_shape=window_size, axis=0)
    errors = torch.mean(
        (reconstructed[:, -1, :] - batch[:, -1, :]) ** 2, dim=1)
```

ENTRADA / SAÍDA o modelo treinado e a matriz padronizada (n_amostras, 38) → vetor 1-D com um erro por janela, de tamanho n_amostras - 50 + 1.

NUMPY sliding_window_view constrói todas as janelas SEM copiar os dados: é uma visão sobre a mesma memória. Com 28 mil janelas de 50×38, copiar seria caro.

TIPAGEM o parâmetro é tipado como Autoencoder, não como “objeto qualquer”: o contrato exige o modelo certo, e o editor avisa antes de rodar.

RESPONSABILIDADE produz o escore. Não decide o que é anomalia — essa decisão é do evaluation/metrics.py, que compara o escore ao limiar.

Implementação do modelo em PyTorch

```
class Autoencoder(nn.Module):
    def __init__(self, input_dim: int,
                  window_size: int = config.WINDOW_SIZE,
                  hidden_dims: tuple[int, ...] = (64, 32),
                  latent_dim: int = 16) -> None:
        super().__init__()
        self.flat_dim = input_dim * window_size # 38 x 50 = 1900

        encoder_dims = (self.flat_dim, *hidden_dims, latent_dim)
        decoder_dims = (latent_dim, *reversed(hidden_dims), self.flat_dim)
        self.encoder = self._build_mlp(encoder_dims)
        self.decoder = self._build_mlp(decoder_dims)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        flattened = x.reshape(x.shape[0], self.flat_dim)
        latent = self.encoder(flattened)
        reconstructed = self.decoder(latent)
        return reconstructed.reshape(
            x.shape[0], self.window_size, self.input_dim)
```

O FUNIL: a janela é comprimida e reconstruída



FICHA TÉCNICA DO TREINO

ARQUITETURA

MLP simétrico: encoder 1900→64→32→16, decoder espelhado.
Ativação ReLU entre camadas.

PARÂMETROS

250.428 pesos treináveis

FUNÇÃO DE PERDA

nn.MSELoss — erro quadrático médio entre a janela e sua reconstrução

OTIMIZADOR

torch.optim.Adam, learning rate 1e-3

ÉPOCAS

máximo 100, executadas 71 (early stopping, paciência 10)

BATCH SIZE

64 janelas por lote

DEVICE

CPU — get_device(prefer_cuda=False); nenhuma GPU foi necessária

ESTABILIDADE

ReduceLROnPlateau (fator 0,5) + gradient clipping (norma 1,0)

SALVAR / CARREGAR

save_model grava o state_dict em artifacts/autoencoder.pt;
load_model restaura numa instância criada com a mesma arquitetura

Configuração experimental

O QUE QUERÍAMOS DESCOBRIR: a capacidade do modelo muda a qualidade da detecção — ou a alavanca real é o limiar?

Divisão dos dados

Treino 80% / validação 20% do arquivo train, em corte temporal (sem embaralhar). O arquivo test é usado só na avaliação final.

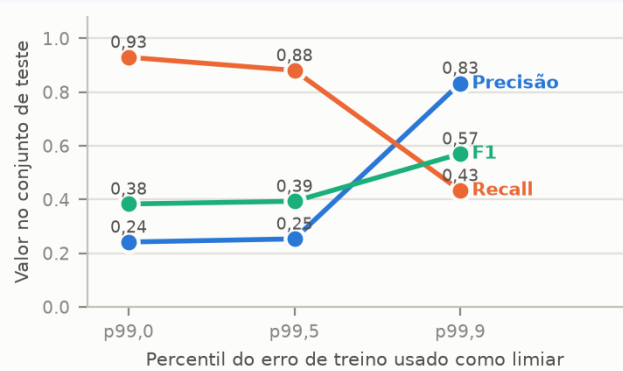
Métricas

Precisão, recall, F1 e acurácia, calculadas em NumPy puro contra o test_label.

Controle de variáveis

Semente fixa (42) e mesmo pré-processamento em todas as execuções: a única coisa que muda entre configurações é o hiperparâmetro em teste.

Config	O que muda	Parâmetros	Épocas	Erro val.	Precisão	Recall	F1
A — Baseline	janela 50 - latente 16 - encoder (64,32)	250.428	71	0,1896	0,253	0,880	0,393
B — Gargalo estreito	janela 50 - latente 8 - encoder (64,32)	249.908	27	0,2090	0,236	0,897	0,374
C — Janela longa	janela 100 - latente 16 - encoder (64,32)	495.528	63	0,2048	0,275	0,964	0,427
D — Maior capacidade	janela 50 - latente 32 - encoder (128,64)	509.196	100	0,1572	0,241	0,928	0,382



DOIS ACHADOS

1. Reconstruir melhor não é detectar melhor. A config D teve o MENOR erro de validação (0,1572) e não lidera a detecção (F1 0,382); a C, com erro de validação pior (0,2048), tem o melhor F1 (0,427) e recall de 96,4%. Capacidade demais reconstrói bem até o que é anômalo — e isso reduz a separação.
2. O limiar move mais que a arquitetura. Entre as quatro configurações o F1 varia 0,054; mexendo só no percentil, na MESMA configuração, varia 0,177 — mais que o triplo. A precisão vai de 25,3% a 83,3%, pagando com recall (88,0% → 43,3%).

Consequência: calibrar o limiar ao custo do alerta é a decisão de maior impacto — e é exatamente a decisão que o RF-06 entrega ao Coordenador.

Testes automatizados e evidências

```
# tests/test_preprocessing.py
class TestPreprocessing(unittest.TestCase):
    def test_standardize_mean(self):
        """A media de cada feature deve ser aprox. zero."""
        X_std, _, _ = standardize(self.X)
        np.testing.assert_allclose(
            np.mean(X_std, axis=0), np.zeros(38), atol=1e-10)

# tests/test_model.py
def test_model_output_shape(self):
    """A saída deve ter o mesmo shape da entrada."""
    output = self.model(self.x)
    self.assertEqual(output.shape, self.x.shape)

def test_load_model_restores_weights(self):
    """load_model restaura pesos idênticos aos salvos."""
    save_model(self.model, self.caminho)
    restaurado = load_model(create_model(38), self.caminho)
```

```
$ python -m unittest discover -s tests
test_standardize_mean ..... ok
test_model_output_shape ..... ok
test_load_model_restores_weights ..... ok
-----
Ran 13 tests in 0.453s
OK
```

O QUE A SUÍTE COBRE

test_data.py — 3 testes

Carregamento devolve DataFrame com 38 colunas · limpeza remove NaN sem alterar colunas · caminho inexistente levanta FileNotFoundError.

test_preprocessing.py — 4 testes

Padronização preserva o shape · média resultante ≈ 0 · desvio ≈ 1 · split respeita as proporções e a ordem.

test_model.py — 4 testes

Saída do modelo tem o shape da entrada · saída é finita (sem NaN/Inf) · save_model cria o arquivo · load_model restaura os pesos idênticos.

test_training.py — 2 testes

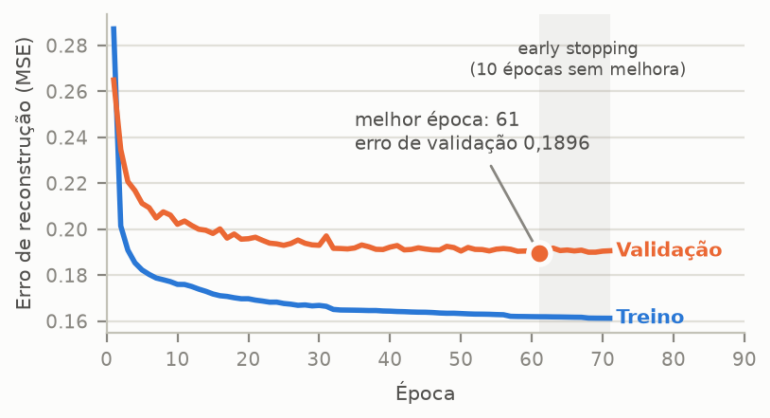
Tensores do DataLoader têm shape (batch, 50, 38) · e dtype float32.

Por que estes testes

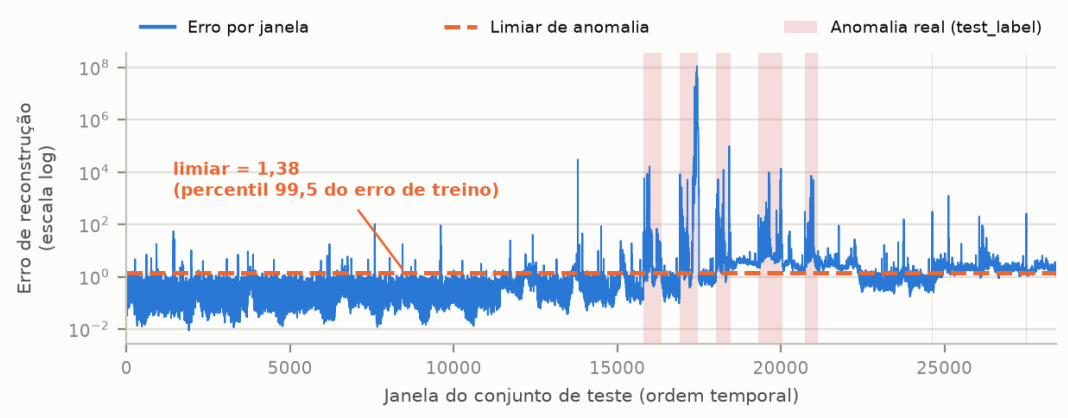
São as fronteiras entre os módulos — shape, dtype e contrato de entrada/saída. É onde a integração quebra silenciosamente quando alguém muda um módulo sem avisar o outro.

Resultados, conclusão e limitações

TREINAMENTO — erro por época



TESTE — erro por janela, limiar e anomalias reais



ERRO DE VALIDAÇÃO	ÉPOCAS / MELHOR	LIMIAR (p99,5)	PRECISÃO · RECALL	CRITÉRIO DE ACEITAÇÃO
0,1896	71 / 61	1,379	25,3% · 88,0%	ATENDIDO

RESOLVEU O PROBLEMA?	LIMITAÇÕES	PRÓXIMOS PASSOS
· Sim, como maquete funcional: o pipeline roda ponta a ponta e o critério de aceitação é atendido. · 5 dos 7 RF atendidos; 2 parciais; 1 é do sistema-alvo. · 3 dos 4 RNF verificados com número medido.	· Erro agregado por janela, não por métrica — trava RF-03 e RNF-02. · Validado só na machine-1-1, com janela de 50. · O baseline de limiar por métrica nunca foi medido: o ganho do ML está argumentado, não provado.	· 1. Decompor o erro por dimensão — destrava RF-03/RNF-02 e é mudança localizada. · 2. Medir o baseline de limiar fixo, para comprovar (ou não) o ganho do modelo. · 3. Testar LSTM-autoencoder e estender às 28 máquinas.